

Python Programmation

CM 5 : File Handling

Julien Bezançon

julien.bezancon@univ-lorraine.fr / <https://julienbez.github.io>



- 1 Before we start...
- 2 Components of a File
- 3 Handling a File in Python
- 4 Navigating Through a File in Python

What did you learn during last lecture ?
Any questions ?

Last Lecture Doggy bag

Functions

- Function definition and calling
- Parameters and arguments
- Variadic arguments
- Python scopes

Advanced strings

- Special characters handling
- String formatting



- 1 Before we start...
- 2 Components of a File**
- 3 Handling a File in Python
- 4 Navigating Through a File in Python

What is a File ?

A file...

- is a document which stores information
- can be hand-made or numeric (or even hand-made, then numerized)
- may have some sort of structure to respect when writing or reading it

Core components of a file

- File format
- File extension
- Character encoding

What is a File Format ?

Wikipedia Definition¹

A file format is the way that information is encoded for storage in a computer file. It may describe the encoding at various levels of abstraction including low-level bit and byte layout as well high-level organization such as markup and tabular structure. A file format may be standardized (which can be proprietary or open) or it can be an ad hoc convention.

File format define the the structure of a file.
It can be standardized or ad hoc.

¹https://en.wikipedia.org/wiki/File_format

What is a Filename Extension ?

Wikipedia Definition²

A filename extension, also known as file name extension or file extension, is a suffix to the name of a computer file that indicates a characteristic of the file contents or its intended use.

Filename extension indicates the format of a file to your computer.
Some examples: .txt, .mp3, .parquet, .json, etc

²https://en.wikipedia.org/wiki/Filename_extension

What is a Character Encoding ?

Wikipedia Definition³

Character encoding is a convention of using a numeric value to represent each character of a writing script. The most popular character encoding on the World Wide Web is UTF-8, which is used in 99.0% of surveyed web sites, as of September 2026.

Character encoding define how each character is encoded in bits and decoded from bits.

³https://en.wikipedia.org/wiki/Character_encoding

Example of Character Encoding

Character Encoding

	C	a	t
ASCII	43	61	74
Unicode	U+0043	U+0061	U+0074
UTF - 8	43	61	74
UTF - 16	00 43	00 61	00 74
UTF - 32	00 00 00 43	00 00 00 61	00 00 00 74

Source: GeeksforGeeks⁴

⁴<https://www.geeksforgeeks.org/computer-science-fundamentals/what-is-character-encoding-system/>

Human VS Machine Readability

Human-readable Formats

- Can be interpreted by a human reading it
- Might be harder to process with machines
- .pdf, .docx, .ods, etc

Machine-readable Formats

- Can not be interpreted by a human reading it
- binary files, parquet files, ...

Some formats are convenient for both humans and machines.

Some examples: .json, .csv, .xml, etc

Describe the following file formats:

- text file format
- comma separated values
- tabulation separated values
- javascript object notation
- parquet file format

More precisely, describe their data structure
and how you would process them.

- 1 Before we start...
- 2 Components of a File
- 3 Handling a File in Python**
- 4 Navigating Through a File in Python

File Opening and Writing (1/2)

```
with open("filepath.txt","r") as f:
    content = f.read() # load filepath.txt content

with open("filepath.txt","w") as f:
    f.write(content) # save content to filepath.txt
```

What would happen if...

- I change the filename extension ?
- I change the file format ?
- I change the encoding of the file ?

File Opening and Writing (2/2)

Changing filename extension

- Will not change anything
- `open()` always read a file as a string (or a list of strings)

Changing file format

- Will not change anything
- Problem: everything will be in a string (data structure is "lost")

Changing the encoding

- May result in incorrect reading of the file
- Solution: use `encoding='your_file_encoding'` in `open()`

First Warning: Encoding

Encoding mistakes are pretty common

- Linux and Mac: UTF-8
- Windows: Unicode or ANSI

How to find and fix them

- Check file encoding before opening it
- Open your file and print some content from it

Second Warning: Pathing

Two kinds of path

- Relative path
 - `my_data_folder/file.txt`
- Absolute path
 - `/home/documents/my_data_folder/file.txt`

Path syntax may vary

- Linux: uses slashes (/) to navigate through folders
- Windows: uses backslashes (\) to navigate through folders

String Parsing

You can use string methods to extract structured data from a string.

How to do it ?

- By understanding the file structure
- By converting the file content to a python data structure

Disadvantages

- Takes time, prone to making errors
- Harder/impossible parsing for some file formats (.json, .parquet, etc)

An Example (.csv)

```
content = """
name,username,id,description
toto,toto78,123,his name is toto
titi,titati,456,he likes cheese
""" # this is a csv file structure

structured = content.split("\n") # separate each line
structured = [i.split(",") for i in structured]

print(structured)
# [["name", "username", "id", "description"],
#  ["toto", "toto78", "123", "his name is toto"],
#  ["titi", "titato", "456", "he likes cheese"]]
```

Another Example (.parquet)

Validation file of the PICO-corpus⁵ in Codium text editor

```
preventio^trial0%.NaalSurgic AdjuvanJB.T.w.Bowel.w.ProjecChP2h.T.h@
Investigators0.TheDoverall50sophylactic%depend)0.n=t=bal.x.#.betwe)1drug0.,0.wh
at000d0foR0were0randoml0assigneE0!_ive0i!&00200m%0/Mday0oePplacebo3follow
ratesac:ly%,ignificantly0.differen8thoserini`ent!0p)0ex0ngcAAl060740versuE06
NInstitut00TumoriN.Mila0Ital00;!!818007560control0armr(respective0Hhoq-0eE_53*50nt
0U0A8Ra0o0Mafm0!0
tence0in0#003
```

Solution: use dedicated libraries

```
import datasets
datasets = load_dataset(
    "cuevascarlos/PICO-breast-cancer", "Default_split-v2")
```

⁵<https://huggingface.co/datasets/cuevascarlos/PICO-breast-cancer>

Dedicated Libraries

Some examples of dedicated libraries

- json: json, ujson, orjson, rapidjson
- csv: csv, pandas, polars, pyarrow, dask
- parquet: parquet, datasets, pyarrow
- xml: xml, lxml, expat, minidom

Disadvantages

- Adds dependencies to your code
- You still have to take a look at the data structure !

Common Practices

Create a folder

```
if not os.path.exists("my_path"):
    os.mkdir("my_path")
```

Create multiple folders

Create a folder

```
if not os.path.exists("my_folder/my_path"):
    os.makedirs("my_folder/my_path")
```

Get all paths of files in a folder

```
import glob
```

```
path_list = [path for path in glob.glob("my_path/*.txt")]
```

Looping Through Files

```
import glob

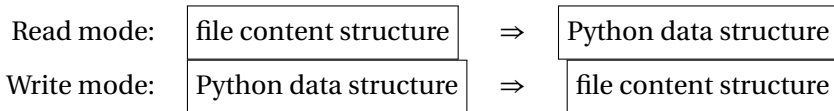
for path in glob.glob("my_path/*.txt"):
    # process the file
```

General tips

- Don't overwrite files - make copies !
- Always make sure the file is well-structured first
- Use `try` and `except` to handle possible errors

- 1 Before we start...
- 2 Components of a File
- 3 Handling a File in Python
- 4 Navigating Through a File in Python**

File structure VS Python structure



Python data structure

- Native structures: lists, tuples, dictionaries, sets
- Custom structures: pandas.Series, array, etc

In Python, most data structures works like
a blending of lists and dictionaries.

An Example (.csv)

```
# lets take structured
# [{"name", "username", "id", "description"},
#  ["toto", "toto78", "123", "his name is toto"],
#  ["titi", "titato", "456", "he likes cheese"]}

# get the username of the first element
username_1 = structured[1][1]

# print every usernames
for i in structured[1:]:
    print(i[1])
```

Another Example (.parquet)

```
import datasets
datasets = load_dataset(
    "cuevascarlos/PICO-breast-cancer", "Default_split-v2")

validation = datasets["validation"]
validation_ids = validation["ids"]

for i in validation_ids:
    print(i)
```

Some Facts

- You will likely ALWAYS find an issue when handling a file
- This is especially true if you are parsing it yourself
- Python data structures can be very confusing, but you get used to it
- Handling a file might not always be hard, but it will ALWAYS takes time

File Issue Handling

```
file = open_file("my_file.txt")
```

```
Traceback (most recent call last):
File "/home/documents/files/How many? (Python).py", line 9, in <module>
fileScan= open(fileName, 'r') #Opens file
FileNotFoundError: [Errno 2] No such file or directory: 'my_file.txt'
```

```
write_file("my_file.txt",content)
```

```
[Errno 13] Permission denied: 'file.txt'
```

File Parsing is HARD

```
content = """
name,username,id,description
toto,toto78,123,his name is toto
titi,titati,456,he likes cheese
""" # this is a csv file structure

structured = content.split("\n") # separate each line
structured = [i.split(",") for i in structured]
```

What happens if...

- I have commas in my content ?
- I use more than one delimiter ?
- I have empty lines ?

Python Data Structure Confusions

```
data = open_json("recipes.json")

print(data[0]["ingredients"][1][1])
print(data[0]["nutrients"]["calories"][2])
# ...
```

Again...

- Take time to understand the structure
- Process step by step

It took me 1h30 - 2h do to the next TD.
I'm very used to handling files in python.

Don't overdo it, do what you can !

Doggy bag

File handling

- Open and write a file
- Encoding and pathing
- String parsing
- Dedicated libraries

File navigation

- File VS python structures
- Common errors

